



LOG2400 – Analyse et conception de logiciels

Hiver 2026

TP No. 4

Groupe 3

2435049 – Nicolas Mihali

2420324 – Akhan Mehmet Sozen

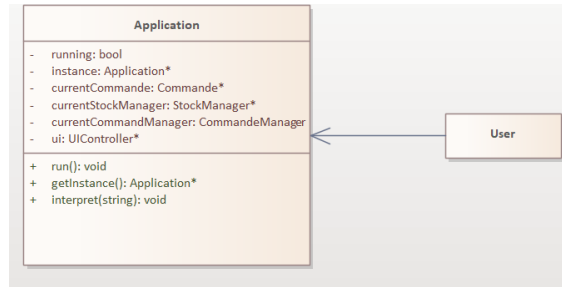
Soumis à : Bryan Alexandre Tavares

Date de la remise : 16 Avril 2026

Les schémas suivants ont obtenu l'approbation de notre chargé de laboratoire en séance.

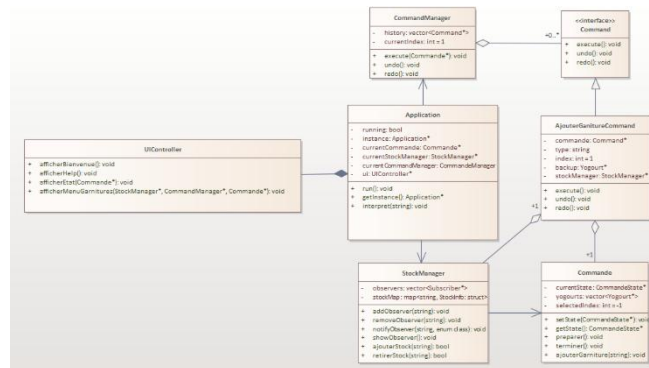
Patron Singleton

Une seule instance de l'application doit exister pour centraliser l'état global (commande courante, stock, historique de commande). Nous avons donc décidé d'utiliser une classe Application comme singleton qui expose un getInstance() statique qui crée une instance unique lors du premier rappel et retourne cette même instance par la suite. Cela permet d'éviter les variables globales.



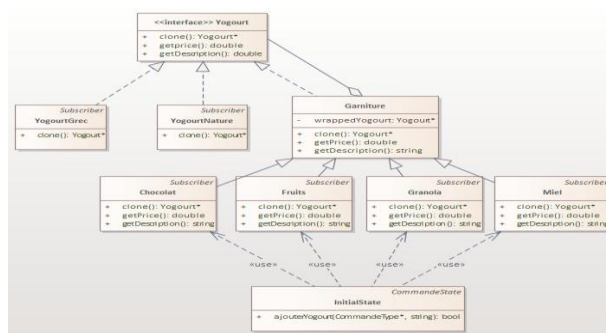
Patron Commande

L'utilisateur doit avoir accès aux fonctions undo et redo pour le choix des garnitures. Nous avons donc implémenté une interface Command qui définit execute(), redo() et undo(). AjouterGranitureCommand encapsule la garniture à ajouter ainsi qu'une sauvegarde de l'état précédent du yogourt avec la variable backup pour permettre l'annulation. CommandManager contient une variable history qui sert d'historique à commande.



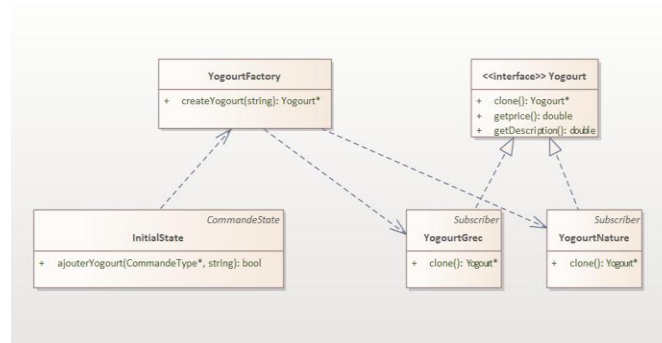
Patron Décorateur

Un yogourt peut recevoir zéro ou plusieurs garnitures dont le prix et la description s'accumulent. Nous avons donc décidé de créer une classe Garniture qui hérite de Yogourt et qui contient aussi un pointeur à un Yogourt (wrappedYogourt). Chaque garniture concrète appelle d'abord l'objet quelle enveloppe et ajoute ensuite sa contribution.



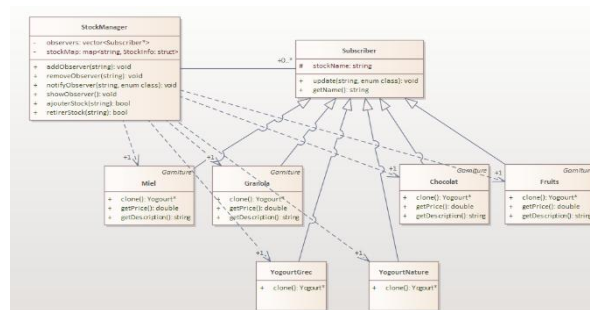
Patron Fabrique

Nous voulons permettre la création d'un yogourt de base (YogourtNature ou YogourtGrec) avec une chaîne de caractères fournie par l'utilisateur. `YogourtFactory::createYogourt(string)` retourne un unique_ptr<Yogourt> selon le type demandé. Cela permet à ce que la logique de construction est isolée du reste de l'application. Grâce à cette implémentation, la logique de construction est isolée du reste de l'application et donc ça facilite l'ajout de nouveaux types de yogourts sans modifier le code client.



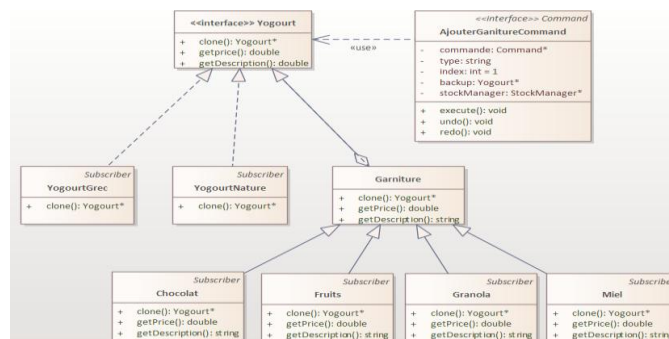
Patron Observateur

Le stock doit pouvoir être géré de façon dynamique. Lorsque le stock d'un ingrédient tombe à 0, ce dernier doit être notifié. Le StockManager joue le rôle de sujet, il maintient une liste de Subscriber et appelle notifyObservers() à chaque modification de stock. Cela permet de découpler le stock des objets qui en dépendent, donc on peut ajouter dynamiquement des nouveaux abonnés sans avoir à modifier StockManager.



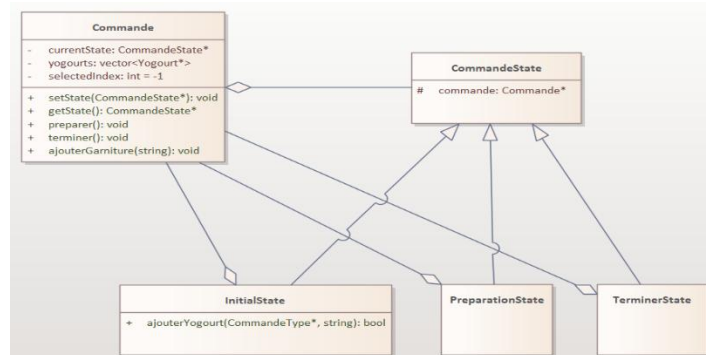
Patron Prototype

Il faut que AjouterGarnitureCommand puisse sauvegarder l'état du yogourt avant une modification pour pouvoir restaurer l'état précédant à l'aide de undo. On a donc une interface Yogourt qui déclare clone(). Chaque classe concrète (yogourts et garniture) fournit une implémentation qui copie l'objet et les décorateurs. On peut ainsi dupliquer un objet sans connaître son type concret et donc cela permet au mécanisme undo de bien fonctionner.



Patron État

Le comportement de la commande doit varier selon son état, donc certaines opérations sont autorisées dans certains états mais interdites dans d'autres. On a donc créé `CommandeState`, une classe abstraite implémentée par `InitialState`, `PreparationState` et `TerminerState`. `Commande` délègue ses opérations à l'état courant et le remplace via `setState()`. Cela permet d'éliminer les blocs conditionnels et les transitions sont rendues plus explicites.



Patron Stratégie

Le calcul du prix final diffère selon le mode de paiement choisi (Prévoyant, Éclair, Poly) et doit pouvoir être changé à l'exécution. On a donc une classe abstraite `ModePaiement` avec `calculerPrix(double)`. Les trois stratégies concrètes appliquent chacune leur propre formule. `Commande` détient un unique `_ptr<ModePaiement>` échangeable. Cela fait en sorte qu'ajouter un nouveau mode de paiement ne nécessite pas de modification de `Commande`.

